

PROJECT REPORT

Bibliotheca

A Digital Library Discovery and Reading Platform

Helping library patrons and everyday readers find books, read public-domain works free in the browser, and keep a personal reading shelf.

Submitted by: [Student name]

Roll / Registration No.: [Number]

Department: [Department]

Institution: [College / University]

Project guide: [Guide name]

Academic year: 2026–27

Date of submission: September 2026

Live source code: github.com/KAUSHALCODER123/digital-library

Contents

1. Title of the Project	3
2. Introduction	4
2.1 What the platform does.....	4
3. Background of the Case/Topic	5
3.1 Libraries and the digital shift	5
3.2 Open book data.....	5
3.3 Copyright.....	5
4. Problem Statement.....	6
5. Objectives of the Project.....	7
6. Scope of the Project.....	8
6.1 In scope	8
6.2 Out of scope	8
7. Methodology.....	9
7.1 Development stages	9
7.2 System architecture	9
7.3 Technology stack.....	10
7.4 Key techniques	10
8. Community/Target Group Details.....	11
9. Data Collection and Analysis.....	12
9.1 Data sources.....	12
9.2 Analysis of the external data.....	12
9.3 Measurements	13
10. Activities/Work Conducted	14
10.1 Pages delivered	14
11. Observations and Findings.....	16
11.1 General observations	16
12. Proposed Solutions/Recommendations	18
12.1 Solution delivered	18
12.2 Recommendations for the library.....	18
13. Benefits and Impact on the Community	19
14. Challenges and Limitations	20
14.1 Challenges faced	20
14.2 Limitations.....	20
15. Conclusion.....	21
16. Future Scope	22

17. References	23
18. Photographs/Evidence of Activities	24
18.1 Home page	24
18.2 Search.....	25
18.3 Book details.....	26
18.4 Reader	27
18.5 Shelf and genres.....	27
18.6 On a phone.....	28
Appendix A: Running the Project.....	30
A.1 Requirements	30
A.2 Setup	30
A.3 Quality checks.....	30

1. Title of the Project

Bibliotheca: A Digital Library Discovery and Reading Platform

Bibliotheca is a web application that works like a modern, online card catalog for a local library. A reader can search millions of books, see a detailed page for each one, read public-domain books directly in the browser, and keep a personal shelf of what they want to read, are reading, and have finished. It is built with Next.js, React and TypeScript, uses Supabase for accounts and storage, and draws book information from Google Books, Open Library, the Internet Archive and Project Gutenberg.

Item	Details
Project type	Full-stack web application (community library service)
Primary users	Library patrons, students and general readers; library staff
Platform	Any modern browser on desktop, tablet and phone
Core technologies	Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS 4, Supabase (PostgreSQL + Auth)
Book data sources	Google Books API, Open Library API, Internet Archive, Project Gutenberg (Gutendex)
Repository	github.com/KAUSHALCODER123/digital-library

2. Introduction

Public libraries hold far more knowledge than most people ever discover. Their catalogs are often hard to use on a phone, show little about each book, and stop at telling you whether a copy is on a shelf. At the same time, tens of thousands of classic books are in the public domain and can be read legally and freely online, but they are scattered across several websites that most readers have never heard of.

Bibliotheca brings these pieces together in one calm, easy-to-use place. It combines the metadata of two large book databases, removes duplicate entries, shows rich detail pages, and opens free books in a comfortable in-browser reader with adjustable text size and light, sepia and dark reading themes. Readers do not need an account to search or read; an optional free account keeps their shelf and reviews in sync across devices.

The project was designed as a service to the reading community: it is accessible to keyboard and screen-reader users, works on slow mobile connections, respects copyright strictly, and gives library staff simple tools to highlight books on the home page.

2.1 What the platform does

- **Search:** search by title, author, subject or ISBN, with filters for language, year, rating, genre and free availability.
- **Book details:** cover, description, authors, translators and illustrators, publisher, pages, ISBN, ratings, editions and similar books.
- **Read online:** public-domain books open in an in-app reader; publisher previews use Google's official viewer.
- **Personal shelf:** "Want to read", "Currently reading", "Read" and "Favorites", with reading progress and yearly statistics.
- **Discovery:** new arrivals, trending books, free classics, staff picks, genre pages and author pages.
- **Accounts:** optional sign-up with email or Google; the shelf saved on a device is merged into the account on first sign-in.
- **Staff tools:** library staff can pin books to the home page and see the most searched terms and most shelved books.

3. Background of the Case/Topic

3.1 Libraries and the digital shift

Library catalogs (OPACs) were originally designed for use at a terminal inside the library building. Many still reflect that origin: search requires exact terms, results show limited information, and mobile layouts are an afterthought. Meanwhile, readers have come to expect the ease of commercial book sites: covers, descriptions, ratings, and instant previews.

3.2 Open book data

Several free, public services now make book information and texts available:

Service	What it provides	Role in this project
Google Books API	Search across a very large catalog, descriptions, categories, ratings, cover thumbnails and embeddable previews	Primary search and metadata; publisher previews
Open Library API	Open bibliographic records, editions, subjects, author biographies, covers and trending lists	Second search source; editions, subjects, authors, trending
Internet Archive	Scanned books, including public-domain works, with an embeddable BookReader	Full-text reading of public scans
Project Gutenberg (Gutendex)	More than 70,000 public-domain e-books as plain text, with a JSON catalog API	Full-text reading in the app's own reader

No single service covers everything. Google Books is broad but rate-limits anonymous use and has no full texts for most books. Open Library has excellent editions and subjects but uneven descriptions. Only the Internet Archive and Project Gutenberg offer legally readable full texts. A useful reader-facing product therefore has to combine them carefully.

3.3 Copyright

Books remain under copyright for decades after publication. A library platform must never present a copyrighted book as “free to read”. Bibliotheca shows a “Read now” button only when a source explicitly states that a book is public domain or offers the publisher’s own preview widget. In every other case it offers “Check availability”, linking to WorldCat (to find a physical copy in a nearby library), Open Library borrowing, or Google Books.

4. Problem Statement

Readers in the local community lack a single, trustworthy, accessible place to discover books and read the ones that are legally free. Existing options are fragmented across several websites, often unusable on phones or with assistive technology, and rarely distinguish clearly between books that can be read for free and books that cannot.

Specifically:

1. **Discovery is fragmented:** book information is split across multiple databases with duplicate and inconsistent records.
2. **Free reading is hidden:** public-domain books are available but hard to find and awkward to read on small screens.
3. **Accessibility is poor:** many catalog interfaces are not usable with a keyboard or screen reader, and do not handle non-Latin or right-to-left titles well.
4. **Continuity is missing:** readers cannot easily keep track of what they want to read or where they stopped reading.
5. **Copyright is unclear:** readers cannot tell whether an online copy is legitimate.

Problem to solve: build a fast, accessible, mobile-friendly platform that unifies book discovery from open sources, lets anyone read public-domain books in the browser, keeps a personal shelf, and never misrepresents a copyrighted book as free.

5. Objectives of the Project

1. Build a unified search across Google Books and Open Library that merges and de-duplicates results by ISBN and by normalized title and author.
2. Provide detailed, search-engine-friendly book pages with editions, similar books and reader reviews.
3. Provide an in-browser reader for public-domain books with adjustable text size, typeface, line spacing and light, sepia and dark themes, and that remembers the reader's place.
4. Enforce copyright rules in code so that "Read now" appears only for legally readable books.
5. Let readers keep a personal shelf without an account, and optionally sync it across devices with a free account.
6. Meet WCAG 2.1 AA accessibility guidelines in both light and dark modes, and work well on phones.
7. Stay reliable when an external service is slow or unavailable, with clear messages and automatic fallback to the other source.
8. Give library staff simple tools to curate home-page shelves and see what the community is searching for.
9. Verify the platform with automated unit, end-to-end and accessibility tests.

6. Scope of the Project

6.1 In scope

- Guest browsing, searching and reading with no sign-in required.
- Search with filters (language, year range, minimum rating, genre, free to read) and sorting (relevance, newest, rating, title).
- Numbered pagination by default (shareable links) and an optional continuous-scroll view.
- Book detail, author and genre pages; home page shelves; search suggestions while typing.
- **Three reading methods:** Project Gutenberg text in the app's reader, Internet Archive embed, Google Books preview widget.
- Personal shelf with four lists, reading progress, statistics, and cross-tab synchronisation.
- Accounts (email and password, Google), password reset, profile, data export, and account deletion.
- Staff tools for curated shelves and search analytics.
- Light and dark site themes; sepia reading theme; responsive layouts from 320 px phones to wide desktops.

6.2 Out of scope

- Hosting or redistributing copyrighted texts. The platform only embeds or links to legitimate sources.
- Integration with a specific library's circulation system (loans, holds, fines). This is listed as future work.
- Offline reading through a service worker (a stretch goal, listed as future work).
- Native mobile apps; the responsive website serves phones instead.

7. Methodology

The project followed an iterative, test-driven approach. Each stage produced working, tested software before the next began, and every change was checked against a written list of edge cases.

7.1 Development stages

Stage	Work carried out
1. Planning	Requirements from the project brief; decisions on stack, data sources, reader strategies, database schema and test plan
2. Design system	Warm “reading room” visual language: paper and ink colors, forest-green primary, brass highlights, Fraunces and Manrope typefaces, catalog-card motif; separate light, dark and sepia themes
3. Data layer	Clients for four book services, a single normalized book type, de-duplication, copyright gate, timeouts and fallbacks; covered by unit tests first
4. Pages	Home, search, book detail, reader, shelf, genres, authors, accounts, staff tools, error and not-found pages
5. Accounts and storage	Supabase schema with row-level security, sign-in flows, shelf synchronisation and merge logic
6. Verification	End-to-end tests on desktop Chrome and mobile Safari (WebKit), automated accessibility scans, manual review with screenshots
7. Hardening	Fixes for issues found in testing (see Section 11), production build, deployment preparation

7.2 System architecture

The browser talks only to the Next.js server and to Supabase. The Next.js server fetches book data from the external services, caches responses for an hour, and keeps any API keys private. Supabase stores accounts, shelves, reviews and staff curation, protected by row-level security policies so each reader can only read and change their own data.

Layer	Components
Browser (client)	React components, search box with suggestions, reader, shelf store (Zustand, saved in the browser), theme switch
Next.js server	Server-rendered pages, /api/search and /api/gutenberg routes, request-level caching, SEO metadata and structured data
Book data layer	Google Books, Open Library, Internet Archive and Gutendex clients; normalization; de-duplication; copyright gate; per-host circuit breaker

Layer	Components
Supabase	PostgreSQL tables: profiles, shelf_items, reviews, curated_items, search_log; Auth (email, Google); row-level security; SQL functions

7.3 Technology stack

Concern	Choice	Reason
Framework	Next.js 16, React 19, TypeScript (strict)	Server rendering for speed and SEO; type safety
Styling	Tailwind CSS 4 with design tokens	One consistent token set for colors, type and spacing
Accessible widgets	Radix UI primitives	Keyboard and screen-reader support for menus, dialogs, tabs
State	Zustand with persistence	Shelf works offline in the browser and syncs when signed in
Validation	Zod	Every URL parameter and form input is checked
Accounts and data	Supabase (PostgreSQL, Auth)	Managed database with row-level security
Testing	Vitest, Testing Library, Playwright, axe-core	Unit, browser and accessibility tests

7.4 Key techniques

- **De-duplication:** records are matched by ISBN-13 first, then ISBN-10 (converted to ISBN-13), then by normalized title plus first initial and surname. Matching records are merged field by field, keeping the richer value (longer description, larger cover, rating with more votes).
- **Copyright gate:** a single function decides whether a book can be read, based only on explicit signals from the source (Gutenberg “copyright: false”, Open Library “public” scans confirmed by the Internet Archive, or Google “embeddable” previews).
- **Resilience:** every external request has a timeout; a failing source is skipped automatically and the page says so; repeated failures open a short circuit breaker so the site stays fast.
- **Reader pagination:** the book is split into chapter-sized sections and each section is laid out in CSS columns, which keeps page turns instant even for very long books.
- **Guest-to-account merge:** when a guest signs in, their device shelf is merged into their account (union of books, newest status wins, favorites kept, furthest progress kept).

8. Community/Target Group Details

The platform is designed for the patrons of a local public library and for general readers in the surrounding community. The groups below shaped specific design decisions.

Group	Needs	How Bibliotheca responds
Library patrons	Find books quickly; know whether they can read them now	Unified search, clear “Read free” badges, links to nearby library holdings (WorldCat)
Students	Free access to classic texts for coursework	Thousands of public-domain classics readable in the browser, no sign-in
Readers on phones and slow connections	Pages that load fast and fit small screens	Server rendering, cached data, skeleton loading states, mobile bottom-sheet filters, reader sized to the phone screen
Readers with visual or motor impairments	Keyboard and screen-reader support; good contrast	WCAG 2.1 AA checks, skip link, labelled controls, keyboard page turns, adjustable text size, dark and sepia themes
Multilingual readers	Correct display of non-Latin and right-to-left titles	Language tags and automatic text direction on titles and descriptions
Older readers	Large, readable text and simple navigation	Large type scale, generous spacing, plain-language labels
Shared-computer users (library terminals)	Privacy between users	The shelf is cleared from the device on sign-out
Library staff	Promote books; understand demand	Curated home shelves; top searches and most-shelved reports

No formal survey of community members was carried out during this phase. The groups above were identified from the project brief and from common accessibility and library-service guidelines. A user study is recommended as the first item of future work (Section 16).

9. Data Collection and Analysis

9.1 Data sources

The platform does not store book metadata of its own. It collects data live from public APIs and stores only what readers create (shelves, reviews) and what staff curate.

Data	Source	Stored by Bibliotheca?
Book metadata, covers, ratings	Google Books, Open Library	No (cached for about one hour on the server)
Full texts	Project Gutenberg, Internet Archive	No (read directly from the source)
Shelf entries and reading progress	Readers	Yes: in the browser for guests; in Supabase for signed-in readers
Reviews and ratings by readers	Readers	Yes, in Supabase
Curated shelves	Library staff	Yes, in Supabase
Search terms (anonymous)	Searches	Yes, for staff reports; ISBNs and e-mail-like text are skipped

9.2 Analysis of the external data

During development the responses of each service were examined to understand their quality and limits. Key findings:

- **Google Books rate limits:** anonymous requests regularly returned HTTP 429 (“too many requests”). The platform therefore falls back to Open Library automatically, and an API key is recommended for production.
- **Duplicate records:** the same work appears in both databases, often with different subtitles or author spellings. Matching by ISBN catches most duplicates; title-and-author matching catches the rest.
- **Mixed scans on Open Library:** a work marked “public” can list many scans, some of which are lending-only. For “Pride and Prejudice”, the Internet Archive confirmed which scans are unrestricted, and the platform picks the most-read public one.
- **Plain-text markup:** Gutenberg texts include licence headers, “[Illustration]” placeholders and underscore italics, which the platform removes before display.
- **Data quality:** some records have implausible dates (for example, a first publication year of 2098) or missing fields; the platform ignores invalid values and hides empty sections rather than printing “undefined”.

9.3 Measurements

The following figures were measured on the development machine during testing.

Measure	Result
Unit tests (data layer, shelf logic, reader maths, security helpers)	110 tests in 12 files, all passing
End-to-end tests per run (desktop Chrome + mobile Safari/WebKit)	103 passing checks per full run; the suite was run twice in a row with no failures (206/206)
Automated accessibility scans (axe-core, WCAG 2.1 A/AA)	9 key pages in light and dark themes plus all three reader themes: 0 serious or critical issues
Book detail page, cached	About 0.3 seconds
Book detail page, first visit (several sources combined)	About 7 seconds (reduced from about 10 seconds by shorter enrichment timeouts and a circuit breaker)
Search results, first visit with Google rate-limited	About 4 seconds (Open Library results shown with a notice)
Code size	About 11,300 lines of TypeScript across 160 tracked files

10. Activities/Work Conducted

Activity	Outcome
Requirement analysis and planning	Written plan covering stack, design, data sources, database, edge cases and tests
Design system	Color tokens for light and dark themes, type scale, catalog-card and bookshelf motifs, designed placeholder covers
Book data integration	Clients for Google Books, Open Library, Internet Archive and Gutendex; normalization; de-duplication; copyright gate
Search experience	Suggestions while typing, filters, sorting, numbered pages, continuous scroll, ISBN lookup, friendly empty and error states
Book detail pages	Metadata, expandable descriptions, contributor roles, editions table, similar books, reviews, structured data for search engines
Reader	Paginated Gutenberg reader with themes and settings; Internet Archive and Google Books embeds with fallbacks
Shelf and accounts	Guest shelf, statistics, account sign-up and sign-in, merge on first sign-in, data export, account deletion
Database setup	Supabase migration applied: five tables with row-level security and SQL functions for reviews, analytics and account deletion
Staff tools	Curated shelves (add, reorder, remove) and analytics tables
Testing and fixes	Unit, end-to-end and accessibility suites; issues found and fixed (see Section 11)
Documentation	This report, a README with setup steps, and screenshots of every main screen

10.1 Pages delivered

Page	Address	Purpose
Home	/	Search card, new arrivals, free classics, genres, trending, staff picks
Search	/search	Results with filters, sorting and paging
Book	/books/{title}--{id}	Full details, reading options, editions, similar books, reviews
Reader	/read/{title}--{id}	Full-screen reading
My shelf	/shelf	Four shelves, progress and statistics
Genres	/genres, /genres/{genre}	Browse by genre with filters

Page	Address	Purpose
Author	/authors/{name}	Biography and books by an author
Accounts	/login, /register, /account	Sign in, sign up, settings, export and deletion
Staff	/admin	Curated shelves and analytics (staff only)

11. Observations and Findings

Testing uncovered a number of real problems. Each was fixed and, where possible, protected by a test so it cannot return.

Finding	Cause	Fix
The in-app reader froze on long books	An entire novel (about 700 KB of text) was laid out at once in columns	The book is now paginated one chapter-sized section at a time
“Dune” was merged with “Dune: House Harkonnen”	Title matching compared surnames only (Frank vs Brian Herbert)	Matching now uses first initial plus surname; regression test added
Detail pages took about 10 seconds	A slow, rate-limited service held up optional enrichment	Shorter timeouts for optional data and a per-host circuit breaker
A cover image showed as a blank box	Slow archive.org images; cached images finishing before the page became interactive	A designed placeholder cover sits underneath until the image arrives
Icons in some buttons shrank to 6 pixels	Conflicting padding classes on the same element	Class merging added so later styles reliably win
The phone header was 6 pixels too wide	Account button and a loading placeholder crowded the header	Account options moved into the mobile menu
Review counts failed contrast checks	Text drawn at 80% opacity	Full-strength muted color used instead
Loading placeholders broke an ARIA rule	Labels on plain containers	Placeholders now announce themselves as status regions
Reader progress looked unchanged between two pages	The last page of one section and the first page of the next shared one position	Position now marks where each page starts
Typing before a page finished loading was lost	The search box was reset when the page became interactive	Text typed early is kept

11.1 General observations

- **Combining two sources clearly improves results:** Open Library adds editions and subjects, and Google Books adds descriptions and previews.
- **A strict copyright gate is essential:** many “public” records point to lending-only scans unless they are verified.

- Accessibility and mobile checks found issues that visual review on a desktop missed.
- Clear, honest messages (“Google Books didn’t respond, so these results come from Open Library only”) keep the site usable during outages.

12. Proposed Solutions/Recommendations

12.1 Solution delivered

Bibliotheca addresses each part of the problem statement:

Problem	Solution in Bibliotheca
Fragmented discovery	One search across Google Books and Open Library, merged and de-duplicated
Hidden free reading	“Read free” badges, a free-to-read filter, a classics shelf and an in-app reader
Poor accessibility	WCAG 2.1 AA checks, keyboard support, screen-reader labels, adjustable reading settings
No reading continuity	Personal shelf, remembered reading position, optional account sync
Unclear copyright	Copyright gate in code; “Check availability” links for everything else

12.2 Recommendations for the library

1. Obtain a Google Books API key to avoid the rate limits observed with anonymous access.
2. Add the Supabase public (anon) key to enable accounts, and configure Google sign-in in the Supabase dashboard.
3. Appoint a staff member to curate “Staff picks” and “New arrivals” monthly using the staff tools.
4. Link the platform from the library’s website and display a QR code in the reading room.
5. Run a short usability study with patrons from each target group and act on the findings.
6. Connect the library’s own catalog (for example through its OPAC or a system such as Koha) so readers can see local copies and place holds.

13. Benefits and Impact on the Community

- **Free access to reading:** thousands of classics can be read at no cost on any device, without an account.
- **Better discovery:** readers find books through search, genres, authors, trending lists and staff recommendations.
- **Inclusion:** the platform is usable with a keyboard or screen reader, in dark or sepia mode, with large text, and with non-Latin scripts.
- **Support for students:** classic texts required for coursework are one search away.
- **Stronger link to the physical library:** every book page points to nearby library copies through WorldCat.
- **Reading habits:** the shelf, progress tracking and yearly statistics encourage regular reading.
- **Privacy by design:** guests need no account; signed-in readers can export or delete all their data; sign-out clears shared computers.
- **Insight for staff:** anonymous search trends and shelving data help the library understand community interests.
- **Low running cost:** the platform relies on free public data sources and a managed database.

14. Challenges and Limitations

14.1 Challenges faced

- **External services:** rate limits, slow responses and occasional outages required timeouts, fallbacks and a circuit breaker.
- **Inconsistent data:** different sources describe the same book differently, which made de-duplication and merging a careful balance.
- **Copyright verification:** deciding which scans are truly public needed an extra check with the Internet Archive.
- **Reader performance:** laying out whole books in the browser was too slow and had to be redesigned.
- **Framework changes:** Next.js 16 introduced changes (asynchronous route parameters, “proxy” instead of middleware, new caching and image rules) that had to be followed carefully.

14.2 Limitations

- Result totals are estimates reported by the sources, and filters such as year and rating are applied page by page.
- Google Books previews depend on each publisher; many books have no preview.
- Internet Archive and Google Books readers cannot report reading progress back to the platform, so progress is tracked only in the Gutenberg reader.
- There is no offline reading yet.
- Accounts require the Supabase public key and, for Google sign-in, provider configuration.
- No formal study with community members has been done yet.

15. Conclusion

Bibliotheca shows that a small team can build a library discovery and reading platform of professional quality on top of free, open book data. It unifies search across two large catalogs, removes duplicates, presents each book clearly, and opens legally free books in a comfortable reader, while never presenting a copyrighted book as free.

The platform works for guests and signed-in readers alike, is usable on phones and with assistive technology, and continues to work when an external service fails. Its behaviour is verified by 110 unit tests, 103 end-to-end checks on desktop and mobile browsers, and automated accessibility scans with no serious issues. With an API key, the Supabase public key and a short usability study, it is ready to serve a local library's readers.

16. Future Scope

- **User study:** interviews and task-based testing with patrons from each target group.
- **Library system integration:** show local availability and allow holds through the library's own catalog.
- **Offline reading:** a service worker so previously opened books and pages work without a connection.
- **Audiobooks:** public-domain recordings from LibriVox alongside the texts.
- **Recommendations:** suggestions based on a reader's shelf and reviews.
- **Reading goals and clubs:** yearly goals, shared lists and book-club pages for community groups.
- **Multilingual interface:** translating the interface into the languages spoken in the community.
- **Richer reader:** bookmarks, highlights, notes and a table of contents.
- **Staff analytics:** trends over time and exports for library reports.

17. References

- [1] Google. Google Books APIs: Using the API. <https://developers.google.com/books/docs/v1/using>
- [2] Internet Archive. Open Library Developers: APIs (Search, Works, Editions, Authors, Covers). <https://openlibrary.org/developers/api>
- [3] Internet Archive. Advanced Search and the BookReader embed. <https://archive.org/advancedsearch.php>
- [4] Gutendex. A JSON web API for Project Gutenberg e-book metadata. <https://gutendex.com>
- [5] Project Gutenberg. Terms of Use and the Project Gutenberg License. <https://www.gutenberg.org/policy/license.html>
- [6] Vercel. Next.js Documentation (version 16). <https://nextjs.org/docs>
- [7] Meta Open Source. React Documentation. <https://react.dev>
- [8] Supabase. Documentation: Auth, Row Level Security and server-side rendering. <https://supabase.com/docs>
- [9] World Wide Web Consortium (W3C). Web Content Accessibility Guidelines (WCAG) 2.1. <https://www.w3.org/TR/WCAG21/>
- [10] W3C. ARIA Authoring Practices Guide: Combobox and Tabs patterns. <https://www.w3.org/WAI/ARIA/apg/>
- [11] Deque Systems. axe-core accessibility testing engine. <https://github.com/dequelabs/axe-core>
- [12] Microsoft. Playwright end-to-end testing documentation. <https://playwright.dev>
- [13] Tailwind Labs. Tailwind CSS v4 documentation. <https://tailwindcss.com/docs>
- [14] Schema.org. Book type (structured data). <https://schema.org/Book>
- [15] OCLC. WorldCat: find items in libraries near you. <https://search.worldcat.org>

18. Photographs/Evidence of Activities

The screenshots below were captured from the running application with live data from the book services. They show the main screens on desktop and on a phone, in light and dark modes.

18.1 Home page

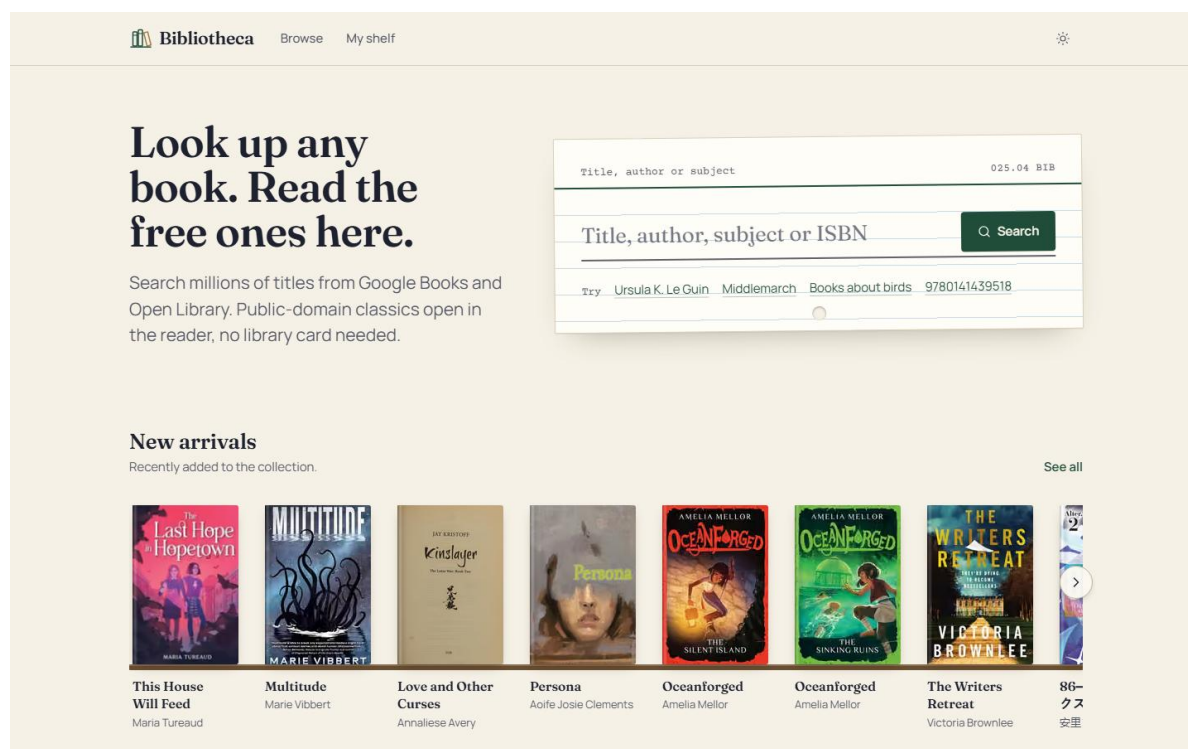


Figure 1: Home page in light mode: the catalog-card search and the “New arrivals” shelf

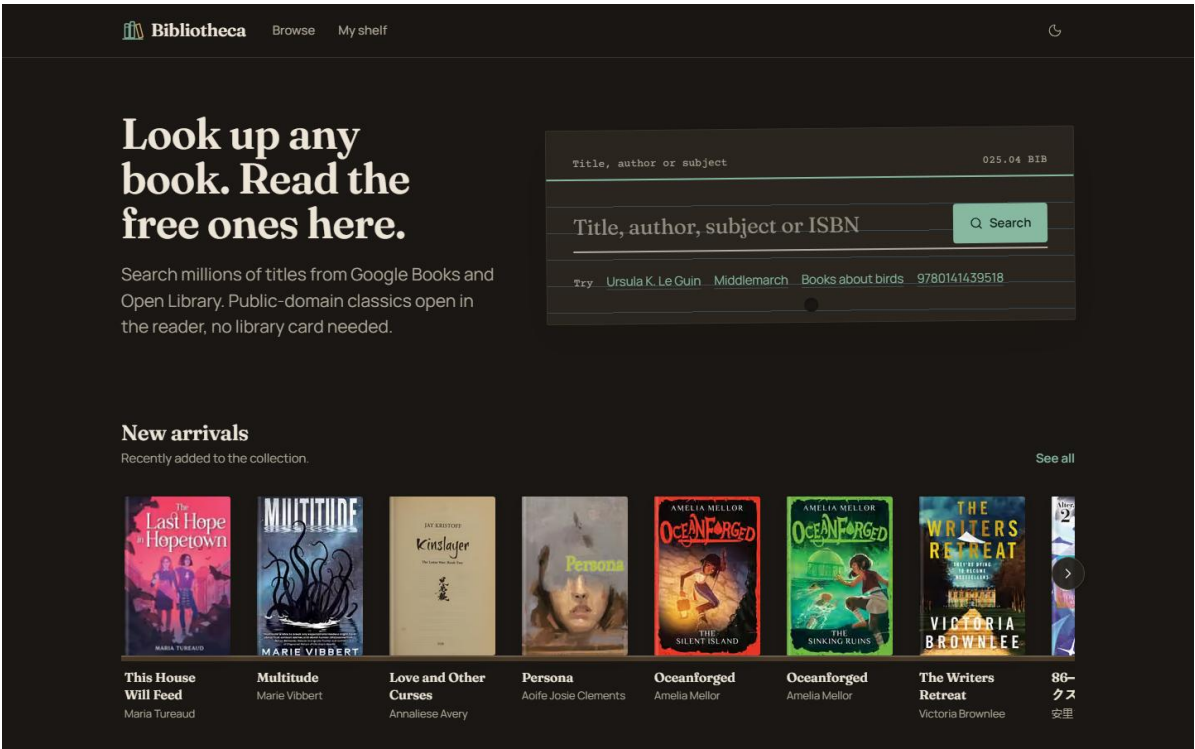


Figure 2: Home page in dark mode

18.2 Search

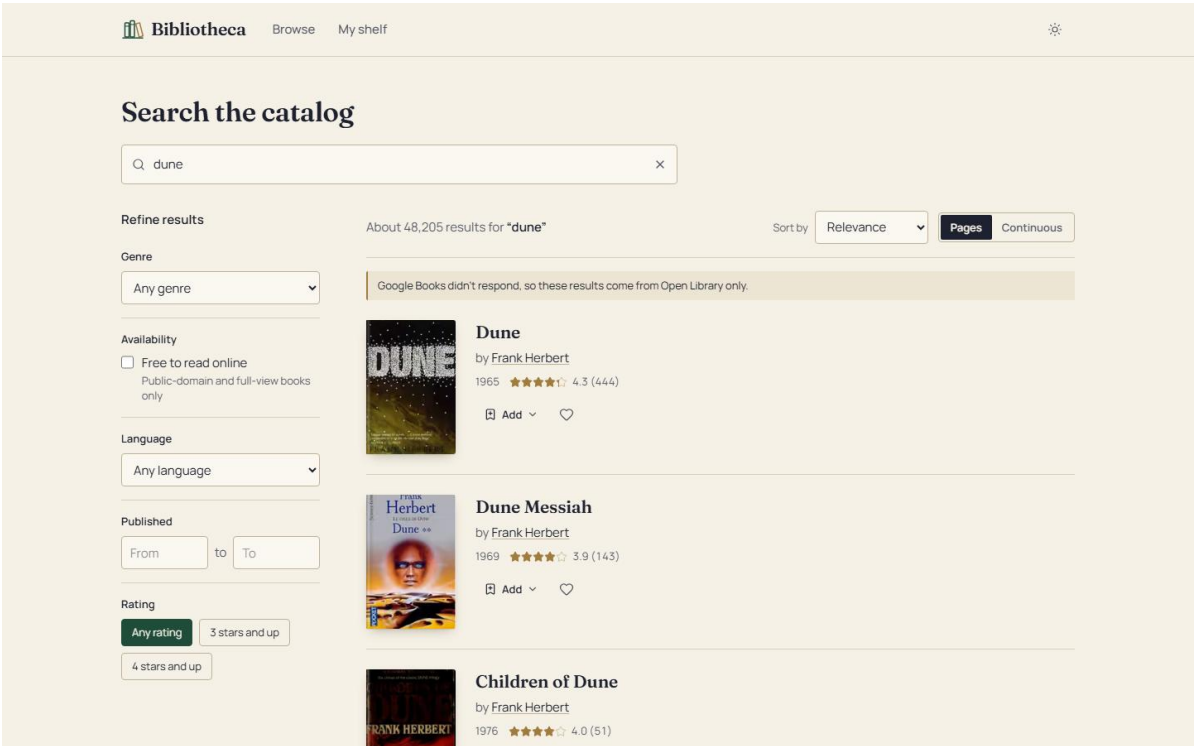


Figure 3: Search results for "dune" with filters, sorting and an honest notice when one source is unavailable

18.3 Book details

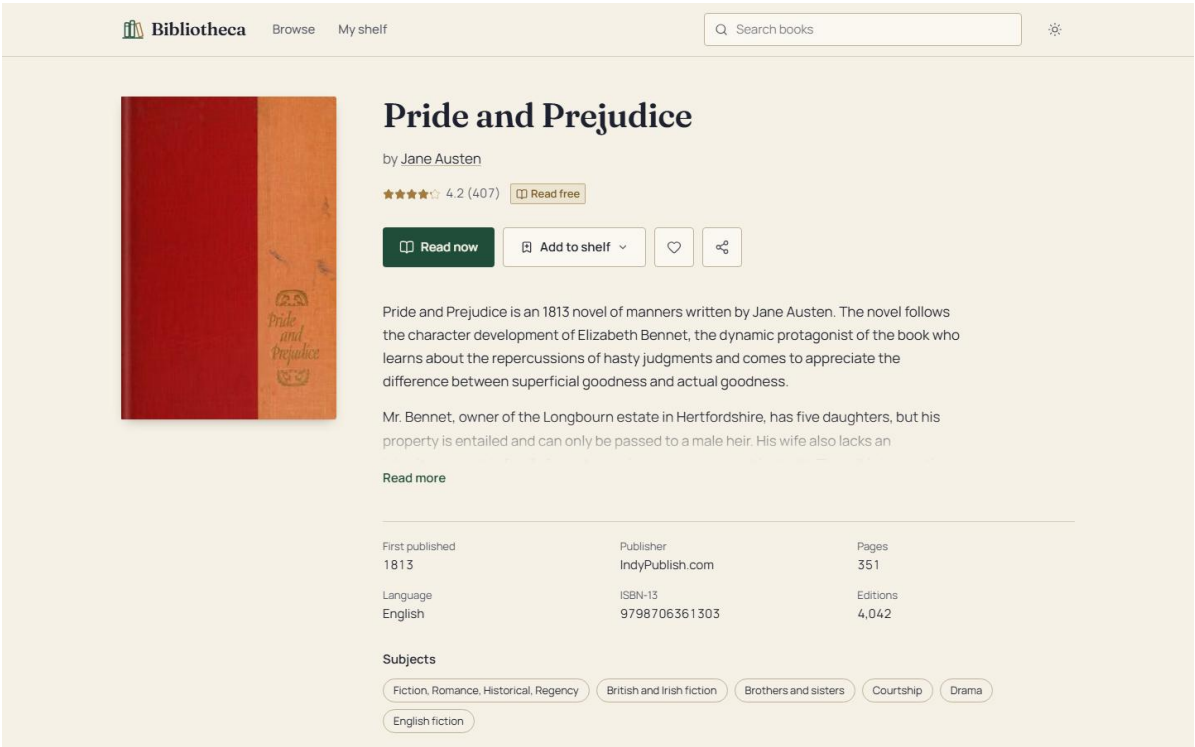


Figure 4: Book page for “Pride and Prejudice” with the “Read now” button and “Read free” badge

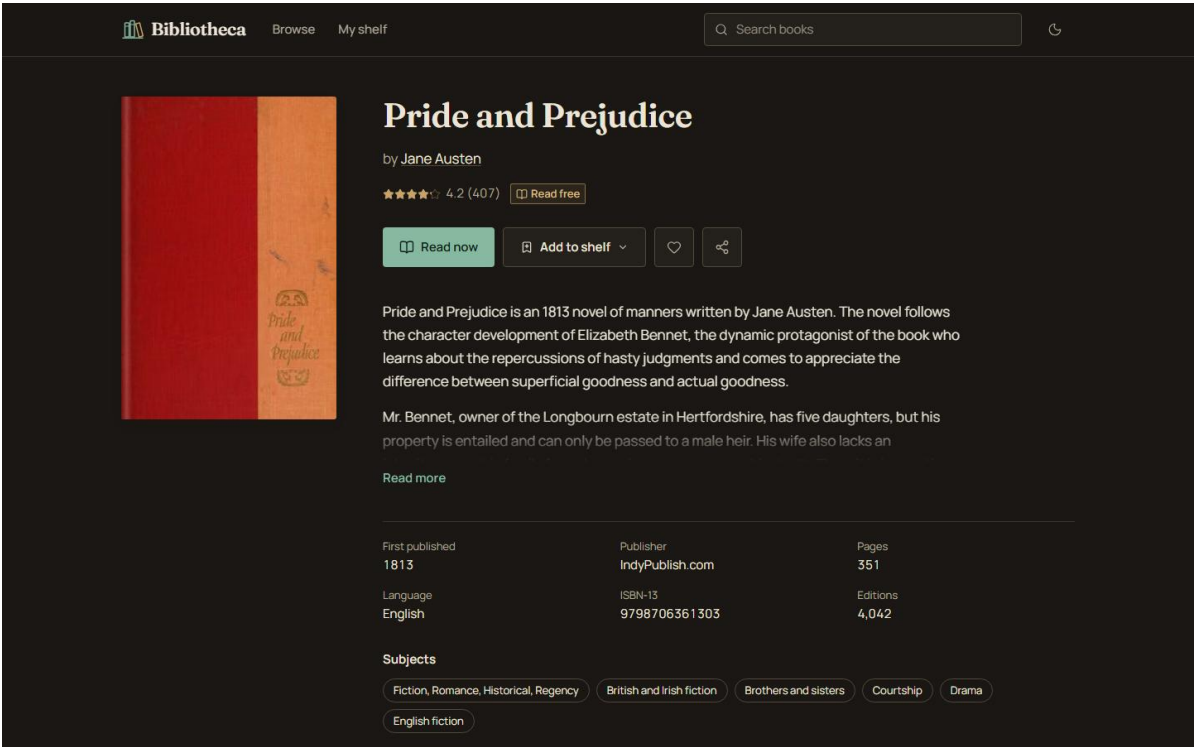


Figure 5: The same book page in dark mode

18.4 Reader



Figure 6: In-app reader in the sepia theme, with progress through the book

18.5 Shelf and genres

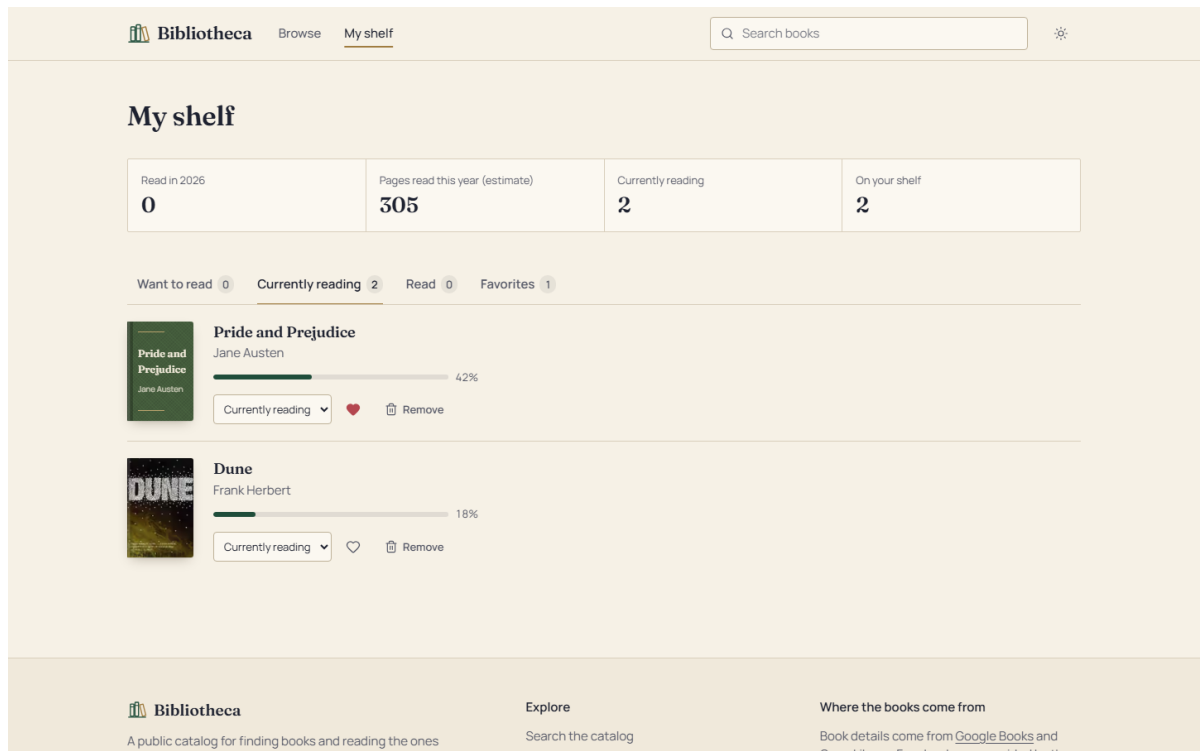


Figure 7: My shelf: statistics, shelves and reading progress

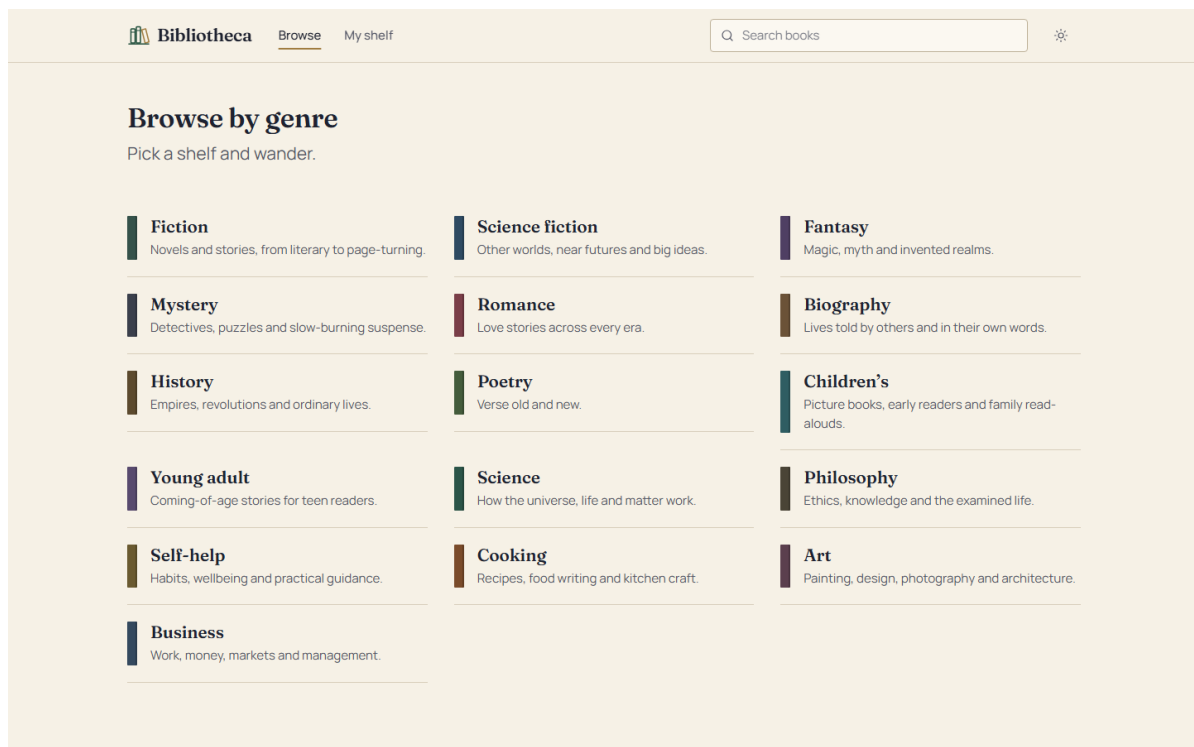


Figure 8: Browse by genre

18.6 On a phone

The phone screenshots below were captured at iPhone 13 size (390 × 844 points).

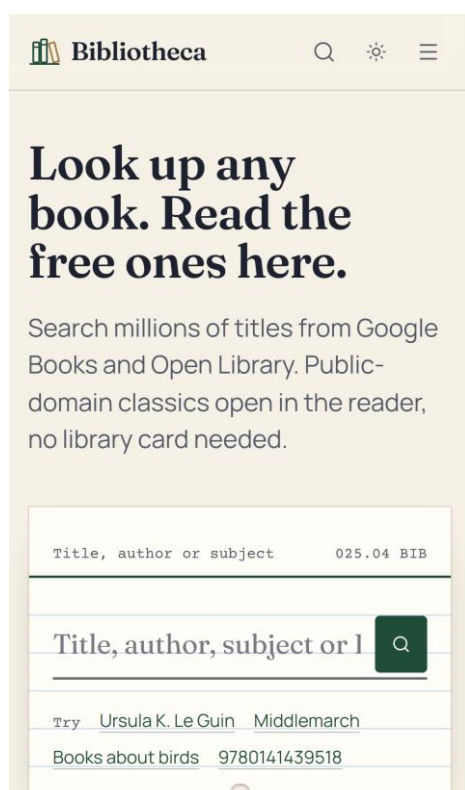


Figure 9: Home page on a phone

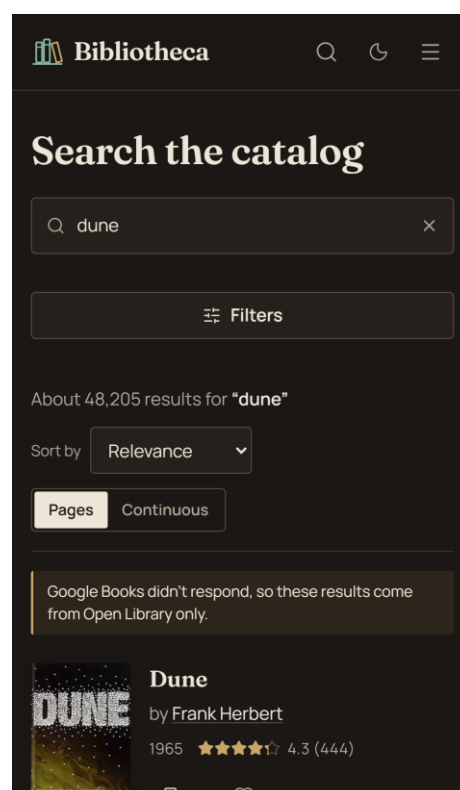


Figure 10: Search on a phone, dark mode

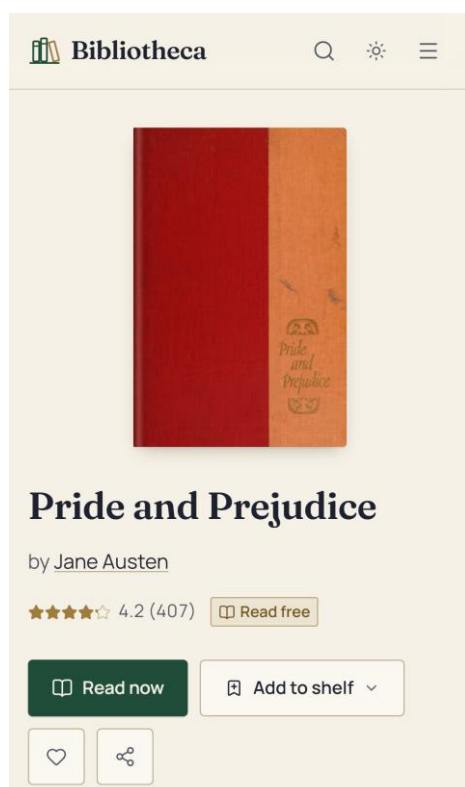


Figure 11: Book page on a phone

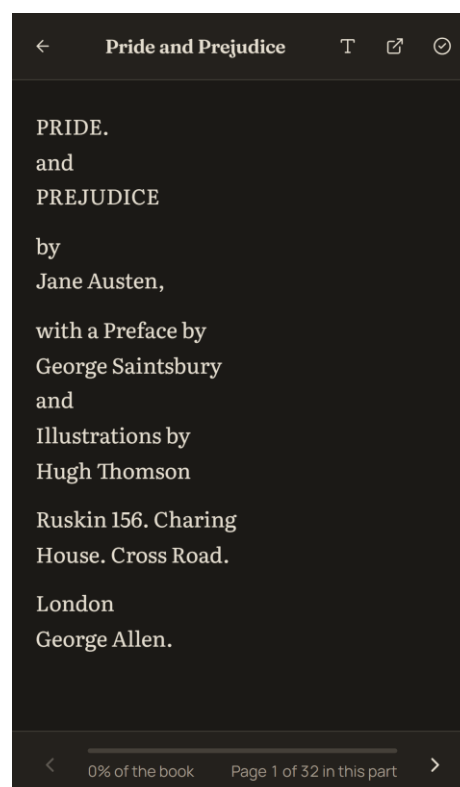


Figure 12: Reader on a phone, dark theme

Appendix A: Running the Project

A.1 Requirements

- Node.js 20 or newer (tested with Node 24)
- A Supabase project (for accounts); optional Google Books API key

A.2 Setup

1. Clone the repository and run “npm install”.
2. Copy “.env.example” to “.env.local” and fill in NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY and, optionally, GOOGLE_BOOKS_API_KEY.
3. Apply the database schema with “npm run db:push” (uses SUPABASE_DB_URL from .env.local).
4. Start the development server with “npm run dev” and open http://localhost:3000.
5. **To make a user a staff member, run in the Supabase SQL editor:** update public.profiles set role = 'staff' where id = '<user id>';

A.3 Quality checks

Command	What it checks
npm run typecheck	TypeScript types (strict mode)
npm run lint	Code quality rules
npm run test	Unit tests (Vitest)
npm run test:e2e	End-to-end and accessibility tests on desktop Chrome and mobile Safari (Playwright)
npm run build	Production build